

Examen Final : Framework Symfony

ESGI 4

Date : 27/07/26

Mini-projet d'application web Symfony

Seul/e ou à deux, vous devez développer (sur 2 jours) un *petit* projet (**système/problème à résoudre au choix**) d'application web avec **Symfony** en respectant [les contraintes données](#).

Vous **présenterez le projet** en l'état, au cours d'une courte présentation de **5 minutes** et **fournirez l'accès à vos sources**, via un dépôt git public.

Comment rendre votre travail

Merci de **lire attentivement** les consignes !

1. **Déposer** votre travail **sur un dépôt git public** (GitHub, Gitlab, BitKeeper, etc.) et fournir **le lien du dépôt**. Le dépôt contiendra la documentation au format Markdown et les sources du site, ainsi qu'un fichier `README.md`. Le fichier `README.md` **contient** (au moins) les sections suivantes :
 - Les **instructions** pour **lancer le projet**,
 - **Documentation** : toutes les ressources de *design* utiles du projet (objectifs, dictionnaire de données, diagrammes, spécifications textuelles, MCD, etc.)
 - (Optionnel) **Commentaires** : section pour libre pour vos retours, difficultés, état d'avancement du projet, ce qui a bien fonctionné ou non, retours sur le module, etc.
2. **Envoyez** votre travail dans **un e-mail** à l'adresse pschuhmacher@myges.fr, **ayant le sujet suivant** :

projet symfony-x-abc

où **x** est la première lettre de votre nom et **abc** votre prénom (j'enverrai donc un e-mail avec le sujet projet symfony-s-paul). Si vous réalisez le groupe par 2, indiquer le nom d'un des membres du groupe.

Dans l'e-mail, **renseigner** :

- **L'URL de votre dépôt où trouver votre travail**,
- **Les membres du groupe (nom et prénom)**.

1 point est réservé au bon respect des consignes de rendu. Merci de **vérifier que votre dépôt est bien public** !

Notation

- **Cahier des charges et respect des contraintes** : l'application développée **respecte les contraintes** fournies, l'état d'avancement du projet est *en accord avec le temps imparti*, l'application **fonctionne** même dans un mode dégradé (**12 points**)
- **Présentation et réponses aux questions et maîtrise du *framework* Symfony**. Le projet et ses **objectifs sont bien présentés**, le code présenté est **compris**, les grands **principes** du framework abordés en cours sont **maîtrisés** (**5 points**)
- **Sources et livrables** : le dépôt contenant les sources est **rendu**, le README et les éléments de documentation demandés sont présents (**3 points**)

Contraintes à respecter (cahier des charges)

L'application doit :

- Être destinée aux humains et **consultable depuis un navigateur web** (réponses HTML, méthodes GET et POST, cookies) ;
- Exposer au moins **deux ressources** (URL+Méthode HTTP)
- Exposer et traiter au moins **un formulaire HTML** (en dehors des formulaires en lien avec les activités d'authentification)
- Utiliser des **assets** JavaScript, CSS et une ou plusieurs images. Usage minimal
- Utiliser une **base de données relationnelle** PostgreSQL pour persister les données de l'application. La base de données sera manipulée via les [Entités de Doctrine](#)
- Utiliser le moteur de templates [Twig](#) pour générer les pages web
- Proposer l'**authentification** des utilisateurs
- Utiliser des **services** (toute ou une partie de la logique métier est déplacée des classes *services* dédiées)

Ce sont des contraintes minimales

Ressources utiles

Synthèse de ressources utiles pour mener à bien le projet.

Contrôleurs/Routing/Validation

- [Routing](#), documentation officielle sur la mise en place du routing dans Symfony;
- [Controller](#), tout savoir sur les Contrôleurs Symfony (AbstractController, Redirection, Injection de Services (injection de dépendances), etc.)
- [Enum Requirements](#), la liste des contraintes de validation accessibles par défaut sur les routes (paramètre requirements)

Templates avec Twig et frontend

- [Site officiel du moteur de templates Twig](#)
- [Creating and Using Templates](#), documentation officielle de Symfony;
- [Templating Engines in PHP](#), article de Fabien Potencier;
- [Front-end Tools: Handling CSS & JavaScript](#)
- [Don't try to sanitize input. Escape output.](#), lecture recommandée sur les bonnes stratégies à employer pour sécuriser un contexte web;

Configurer VS Code pour Twig :

- Ajouter **auto-complétion des balises HTML** dans les fichiers `.twig` : *File > Preferences > Settings > Emmet > Include Languages* : ajouter **Item:** twig **Value:** html;
- [Extension Twig Language 2](#), [TWIG pack](#) : syntaxe highlighting, formatting, snippets, etc.

Formulaires

- [Forms](#), documentation sur la création et gestion des formulaires dans une application Symfony
- [The Form Component](#), le composant *Form* fourni par Symfony. Usage, validation, token anti-CSRF, etc.

Services et pattern *Service Container*

- [Service Container](#), documentation officielle sur le composant *Service Container*;
- [Types of Injection](#), documentation officielle sur les différents types d'injection ;
- [Defining Services Dependencies Automatically](#) (Auto-wiring), documentation officielle sur l'autowiring ou chargement automatique de classes
- [Binding Arguments by Name or Type](#), utiliser le mot-clef bind dans `config/services.yaml` pour associer des valeurs/implémentations à des noms d'arguments ou à des types

Doctrine : ORM et entités

- [Databases and the Doctrine ORM](#), on peut suivre cette page de la documentation officielle de Symfony, section après section, pour se familiariser avec l'usage de Doctrine dans une application Symfony. **Lecture recommandée**
- [Getting Started with Doctrine \(tutoriel\)](#), documentation officielle de Doctrine. Guide de démarrage pour bien comprendre l'utilisation de l'ORM. **Lecture recommandée**
- [Doctrine Configuration Reference](#), documentation officielle de Symfony (API), sur la configuration de Doctrine dans un projet Symfony;
- [Extra Lazy Associations](#), optimiser les requêtes SQL lors de la récupération de données prises dans des associations;
- [Pagination](#), résultats paginés avec Doctrine
- [A Query fails, how can I debug it? \(FAQ\)](#)
- [Optimiser l'utilisation de Doctrine](#), un bon article
- [Anemic Domain Model](#), de Martin Fowler. Aller plus loin, sur les entités *anémiques* VS entités *riches* (*Anemic vs rich entities*) et anti-patterns liés à l'usage d'un ORM